

Testing Report

Course: CS 419

Semester: Spring 2026

Team Members: Amanda Lei, Christopher Chan

Summary

This report details the methodology and findings of a comprehensive security assessment and penetration test conducted on the Secure Document Sharing System. The objective of the assessment was to evaluate the application's security posture against the OWASP Top 10 vulnerabilities and project-specific requirements, focusing on authentication, authorization, input validation, session management, and configuration security. Through manual testing and code review, the assessment identified four primary vulnerabilities during the initial development phases: exposure of the symmetric encryption key in version control (Critical), insufficient file upload limits posing a Denial of Service risk (High), an insecure HTTP deployment with debug mode enabled (High), and incomplete browser security headers (Low). All identified vulnerabilities were promptly remediated prior to final deployment. Following the implementation of secure key rotation, strict file size validation, local TLS enforcement, and a comprehensive Content Security Policy, the application demonstrates a robust security posture capable of safely managing and encrypting confidential documents.

Testing Methodology

The penetration testing and security assessment were conducted using a combination of manual testing, static code analysis, and browser-based dynamic analysis. The testing methodology was aligned with the OWASP Top 10 vulnerabilities and the specific security requirements outlined for the Secure Document Sharing System project. We created a test script to initialize encrypted user data with predefined roles, ensuring consistent testing without exposing plaintext credentials --> called test_users.py run using `python3 -m tests.test_users`

Testing was performed locally using a development environment. The assessment focused on five primary categories:

- Authentication & Authorization: Testing bypass techniques, password enforcement, rate limiting, and horizontal/vertical privilege escalation.
- Input Validation: Submitting malformed data, excessively large payloads, and restricted file types to test for injection and Denial of Service (DoS) vulnerabilities.
- Session Management: Analyzing cookie security flags, session timeouts, and fixation protections.
- Configuration: Inspecting HTTP response headers and transport layer security setups using browser developer tools.

Testing Checklist

Authentication Testing

- ☒ Brute force protection
- ☒ Password complexity enforcement
- ☒ Session management
- ☒ Logout functionality
- ☒ Password reset security

Authorization Testing

- ☒ Horizontal privilege escalation
- ☒ Vertical privilege escalation
- ☒ Direct object reference
- ☒ Forced browsing

Input Validation Testing

- ☒ XSS attempts
- ☒ Path traversal
- ☒ Command injection
- ☒ File upload vulnerabilities

Session Security Testing

- ☒ Session fixation
- ☒ Session hijacking
- ☒ Concurrent session handling
- ☒ Session timeout

Configuration Testing

- ☒ Security headers
- ☒ TLS/SSL configuration
- ☒ Error handling
- ☒ Debug mode disabled

Findings

Finding 1: Session Timeout

Severity: Medium

Description: While testing to see if session security testing passed, we noticed the application relied on the browser's cookie directives to expire the session. The SessionManager did not track an inactivity timeout threshold.

Impact: Relied only on client-side timeout means the session will remain on forever in the server's memory. If an attacker compromises a user's session token, they can use that token to maintain unauthorized access forever.

Steps to reproduce:

- 1) Authenticate the application and extract the session_token value from the browser's Developer Tools
- 2) Wait over 30 minutes to make sure the client-side cookie has timed out and been purged.
- 3) Submit a protected endpoint using an external tool and manually injecting the extracted token into the request headers.
- 4) Observe that the server grants access, showing the session is still active on the backend.

Proof of Concept: The original validate_session() function checked for the existence of the token in sessions.json, but lacked the logical checks to compare the current system time against the session's last_activity timestamp

Remediation Recommendations: Upgrade SessionManager to enforce a strict 30-minute idle timeout on the server side. The validate_session() function now calculates the time passed since last_activity. If the value passes 1800 seconds (30 minutes), the server destroys the session record and forces the user to re-login.

Finding 2: Brute Force Protection

Severity: High

Description: During input testing, we noticed that password complexity requirements and email format validation were only enforced on the frontend using HTML5 attributes. The backend /register route lacked corresponding validation logic.

Impact: Relied on client-side validation which provided a security flaw. An attacker could bypass the browser's interface by using an intercepting proxy. This would allow the creation of accounts with weak passwords which leaves accounts vulnerable for brute-force attacks.

Steps to reproduce:

- 1) Open an intercepting proxy and configure the browser to route traffic through it
- 2) Fill out the registration form with valid data to pass the HTML5 checks and click "Register"
- 3) Intercept the POST /register HTTP request before it leaves the proxy
- 4) Modify the intercepted password from a strong password to a simple password
- 5) Forward the request to the server and attempt the account with the new login

Proof of Concept: A direct HTTP POST request omitting the UI constraints resulted in successful account creation: curl -X POST -d

```
"username=testuser&email=test&password=1&confirm_password=1"
```

```
http://127.0.0.1:5000/register
```

Remediation Recommendations: Implemented strict, server-side validation. Added Python regular expressions checks within /register Flask route to independently verify that the email format is valid and the password contains the minimum requirements

Finding 3: Horizontal Privilege Escalation

Severity: High

Description: During authorization testing, it was observed that the initial implementation of the document /delete/<doc_id> endpoint did not properly verify if the authenticated user actually owned the document they were requesting to delete. The server only checked if the document existed in the database.

Impact: An attacker could perform Horizontal Privilege Escalation. By authenticating as a standard user and guessing or intercepting the doc_id of a confidential file belonging to another user, the attacker could submit a direct HTTP POST request to delete that file. This results in unauthorized data destruction and a severe loss of data integrity.

Steps to reproduce:

- 1) Authenticate as UserA.
- 2) Upload a file as UserB to generate a valid doc_id.
- 3) While logged in as UserA, use an interception proxy (like Burp Suite) to send a POST /delete/<UserB_doc_id> request.
- 4) Observe that the server successfully deletes UserB's file from both the physical disk and the documents.json database.

Proof of concept: The original route logic fetched the document (`doc = documents.get(doc_id)`) and immediately proceeded to `del documents[doc_id]` without comparing `doc['owner']` against `g.user['username']`.

Remediation recommendations: Implemented strict access control logic in the endpoint. The route now explicitly evaluates `can_delete = doc['owner'] == g.user['username'] or g.user['role'] == 'admin'`. If this condition is false, the server logs an `ACCESS_DENIED` event and aborts the request with a 403 Forbidden status.

Finding 4: Path Traversal

Severity: Critical

Description: During input validation testing, it was identified that the application processed file retrieval and storage paths insecurely. File names were concatenated directly with the base upload directory without sanitizing directory traversal character sequences.

Impact: An attacker could exploit this to navigate outside the intended `/data/files` directory. By crafting a malicious filename payload, an attacker could read sensitive system files (e.g., `/etc/passwd`), extract the application's source code, or download the `secret.key` file, leading to a total system compromise.

Steps to reproduce:

- 1) Intercept a legitimate document download or upload request.
- 2) Modify the file path parameter to include traversal sequences: `../../../../secret.key`.
- 3) Submit the request and observe that the server retrieves and returns the contents of the targeted out-of-bounds file.

Proof of concept: The vulnerable implementation used `full_path = os.path.join(base_dir, user_input)` without verifying the final resolved path.

Remediation Recommendations: Implemented a robust `safe_file_path` utility function. This function utilizes Werkzeug's `secure_filename` to strip dangerous characters, and further verifies that the absolute path of the requested file strictly begins with the absolute path of the intended `base_dir` using `os.path.abspath()`.

Finding 5: Security Headers

Severity: Medium

Description: During configuration testing, an inspection of the application's HTTP responses revealed a lack of baseline security headers. Crucial headers such as `Content-Security-Policy`,

X-Frame-Options, and Strict-Transport-Security were entirely absent from the initial server configuration.

Impact: The absence of these headers leaves the application and its users vulnerable to a variety of client-side attacks. Without X-Frame-Options, the application is susceptible to Clickjacking (UI redressing). Without a Content-Security-Policy (CSP), the application has no defense-in-depth mechanism to mitigate Cross-Site Scripting (XSS) if input validation fails.

Steps to reproduce:

- 1) Navigate to the application's login page in a modern web browser.
- 2) Open the Developer Tools (Network Tab) and inspect the HTTP response headers for the primary HTML document.
- 3) Run the application through an automated scanner like securityheaders.com.

Proof of concept: The raw HTTP response headers only included standard Flask defaults (e.g., Content-Type: text/html and Content-Length), missing all modern security directives.

Remediation recommendations: Developed and registered a global `@app.after_request` middleware function. This function explicitly injects a comprehensive suite of security headers into every outgoing response, including a strict CSP, X-Frame-Options: DENY, X-Content-Type-Options: nosniff, and HTTP Strict Transport Security (HSTS) . Checklist Updates: If you choose to use these, make sure to update your Section 3 Checklist to mark these specific items as [REMEDIATED]: Horizontal privilege escalation (See Finding 11) Path traversal (See Finding 12) Security headers (See Finding 13) These are incredibly strong findings. If you put these into your final PDF, your professor will know you truly understood the core concepts of CS 419!